Optimization killers

Benjamin Gruenbaum edited this page on May 31, 2018 · 57 revisions

All this is wrong in TurboFan. (Node 8+)

Please do not take anything written here as performance advice, this is here for historical reasons.

Optimization killers

Introduction

This document will contain advice to avoid writing code that will perform significantly worse than expected. Specifically those patterns that cause V8 (relevant to Node.JS, Opera, Chromium...) to refuse to optimize the affected function.

vhf is also working on a similar project that tries to list every killers in V8 Crankshaft Engine: V8 Bailout Reasons.

Some V8 background

In V8 there is no interpreter but there are 2 different compilers: generic and optimizing. That means your JavaScript is always compiled and run directly as native code. That means it's fast, right? Wrong. Code being compiled to native code by itself isn't hugely significant for performance. It just removes interpreter overhead but the code will still be slow if not optimized.

For example in generic compiler `a + b` will become something like this:

```
mov eax, a
mov ebx, b
```

```
call RuntimeAdd
```

In other words it just calls a runtime function. If `a` and `b` will always be integers, then something like this:

```
mov eax, a
mov ebx, b
add eax, ebx
```

would perform massively faster than the call figuring out complex JavaScript addition semantics at runtime.

In general you will get the former kind of code from the generic compiler and the latter kind of code from the optimizing compiler. Code compiled by the optimizing compiler can easily be, say, 100x faster than the code generated by the generic compiler. But there is a catch, you can't just write any kind of JavaScript and have it be optimized. There is a lot of patterns, some even idiomatic, in JS that the optimizing compiler refuses to touch (it "bails out").

It is important to note that patterns that cause optimization bailouts affect the entire containing function. Code is optimized 1 function at a time, without knowledge of what other code is doing (unless that code is being inlined in the function that is being currently optimized).

This guide will cover most patterns that cause the containing function go to "deoptimization hell". They will be subject to change and suggested work-arounds may become unnecessary when the optimizing compiler is updated to recognize more and more patterns.

Topics

1. Tooling
2. Unsupported syntax
3. Managing `arguments`
4. Switch-case
5. For-in
6. Infinite loops with deep logic exit conditions or unclear exit conditions

1. Tooling

You should be able to use Node.js with some V8 flags to verify how patterns affect optimization. Generally you will make a function that contains the pattern, call it with all possible types to feed in the types and then call internal V8 functions to optimize and inspect it:

test.js:

```
//Function that contains the pattern to be inspected (using an `eval` statement)
function exampleFunction() {
    return 3;
    eval('');
}

function printStatus(fn) {
    switch(%GetOptimizationStatus(fn)) {
        case 1: console.log("Function is optimized"); break;
        case 2: console.log("Function is not optimized"); break;
        case 3: console.log("Function is always optimized"); break;
        case 4: console.log("Function is never optimized"); break;
        case 6: console.log("Function is maybe deoptimized"); break;
        case 7: console.log("Function is optimized by TurboFan"); break;
        default: console.log("Unknown optimization status"); break;
    }
}

//Fill type-info
exampleFunction();
// 2 calls are needed to go from uninitialized -> pre-monomorphic -> monomorphic
exampleFunction();

%OptimizeFunctionOnNextCall(exampleFunction);
//The next call
exampleFunction();

//Check
printStatus(exampleFunction);
```

Running it:

```
$ node --trace_opt --trace_deopt --allow-natives-syntax test.js
(v0.12.7) Function is not optimized
(v4.0.0) Function is optimized by TurboFan
```

https://codereview.chromium.org/1962103003

To see it's working, comment out the `eval` statement and re-run:

```
$ node --trace_opt --trace_deopt --allow-natives-syntax test.js
[optimizing 000003FFCBF74231 <JS Function exampleFunction (SharedFunctionInfo 00000000FE
```

```
Function is optimized
```

It is important to use the tooling to verify that the workarounds are working and necessary.

2. Unsupported syntax

Some constructs are flat out not supported in the optimizing compiler and using such syntax will make the containing function unoptimizable.

It is **important to note** that even if the construct is unreachable or not run, these constructs still cause a function to be unoptimizable.

For example this does not help:

```
if (DEVELOPMENT) {
    debugger;
}
```

The above will punish the entire containing function even if the debugger statement is never reached.

Currently not optimizable:

- ~~Generator functions~~ (Optimized in V8 5.7)
- ~~Functions that contain a for-of statement~~ (Optimized in V8 commit 11e1e20)
- ~~Functions that contain a try-catch statement~~ (Optimized in V8 commit 9aac80f / V8 5.3 / node 7.x)
- ~~Functions that contain a try-finally statement~~ (Optimized in V8 commit 9aac80f / V8 5.3 / node 7.x)
- ~~Functions that contain a compound `let` assignment~~ (Optimized as of Chrome 56 / V8 5.6!)
- ~~Functions that contain a compound `const` assignment~~ (Optimized as of Chrome 56 / V8 5.6!)
- Functions that contain object literals that contain `__proto__`, or `get` or `set` declarations.

Likely never optimizable:

- Functions that contain a `debugger` statement
- Functions that call literally `eval()`
- Functions that contain a `with` statement

Just to be clear on the last point: the entire containing function is unavailable for optimization, when you do any of this:

```
function containsObjectLiteralWithProto() {
    return {__proto__: 3};
}
```

```
function containsObjectLiteralWithGetter() {
    return {
        get prop() {
            return 3;
        }
    };
}
```

```
function containsObjectLiteralWithSetter() {
    return {
        set prop(val) {
            this.val = val;
        }
    };
}
```

Direct `eval` and `with` deserve a special mention here because they cause everything in their path to be dynamically scoped, thus possibly corrupting many other functions too as it became impossible to lexically tell to what variables are bound to.

Workarounds

Some of these statements cannot be avoided in production code such as `try-finally` and `try-catch`. To use such statements with minimal impact, they must be isolated to a minimal function so that the main code is not affected:

```
var errorObject = {value: null};
function tryCatch(fn, ctx, args) {
    try {
        return fn.apply(ctx, args);
    }
    catch(e) {
        errorObject.value = e;
        return errorObject;
    }
}
```

```
var result = tryCatch(mightThrow, void 0, [1,2,3]);
//Unambiguously tells whether the call threw
if(result === errorObject) {
    var error = errorObject.value;
}
else {
    //result is the returned value
}
```

3. Managing `arguments`

There are numerous ways to use `arguments` in a way that causes the function to be unoptimizable. One must be extremely careful when using `arguments`.

3.1. Reassigning a defined parameter while also mentioning `arguments` in the body (in sloppy mode only). Typical example:

```
function defaultArgsReassign(a, b) {
     if (arguments.length < 2) b = 5;
}
```

Workaround is to save the parameter to a new variable:

```
function reAssignParam(a, b_) {
    var b = b_;
    //unlike b_, b can safely be reassigned
    if (arguments.length < 2) b = 5;
}
```

If this was the only use case for `arguments` in the function, it can often be replaced with a `undefined` check:

```
function reAssignParam(a, b) {
    if (b === void 0) b = 5;
}
```

If it's likely that the function will later introduce `arguments` then maintenance could easily forget to leave the re-assignent there though.

Workaround 2: enable strict mode (`'use strict'`) per-file or per-function.

3.2. Leaking arguments:

```
function leaksArguments1() {
    return arguments;
}
```

```
function leaksArguments2() {
    var args = [].slice.call(arguments);
}
```

```
function leaksArguments3() {
    var a = arguments;
    return function() {
        return a;
    };
}
```

The `arguments` object must not be passed or leaked anywhere.

Workaround for proxying is to create array in-line:

```
function doesntLeakArguments() {
                    //.length is just an integer, this doesn't leak
                    //the arguments object itself
    var args = new Array(arguments.length);
    for(var i = 0; i < args.length; ++i) {
                //i is always valid index in the arguments object
        args[i] = arguments[i];
    }
    return args;
}

function anotherNotLeakingExample() {
    var i = arguments.length;
    var args = [];
    while (i--) args[i] = arguments[i];
    return args
}
```

It takes a lot of code and is annoying so it might be worth to analyze if it's really worth it. Then again optimizing always takes a lot of code when more code means more explicitly nailed down semantics.

However, if you have a build-step, this can also be achieved with a macro that doesn't necessitate the use of source maps and lets the source code stay valid JavaScript:

```
function doesntLeakArguments() {
    INLINE_SLICE(args, arguments);
    return args;
}
```

The above technique is used in bluebird and the result is expanded into this in the build step:

```
function doesntLeakArguments() {
    var $_len = arguments.length;
    var args = new Array($_len);
    for(var $_i = 0; $_i < $_len; ++$_i) {
        args[$_i] = arguments[$_i];
    }
    return args;
}
```

3.3. Assignment to arguments:

This is actually possible in sloppy mode:

```
function assignToArguments() {
    arguments = 3;
    return arguments;
}
```

Workaround: there is no need to write such idiotic code. In strict mode, it throws an exception anyway.

What is safe `arguments` usage?

Only use

- `arguments.length`
- `arguments[i]` **where `i` is always a valid integer index into the arguments, and can not be out of bound**
- Never use `arguments` directly without `.length` or `[i]`
- STRICTLY `fn.apply(y, arguments)` is ok, nothing else is, e.g. `.slice`. `Function#apply` is special.

- Be aware that adding properties to functions (e.g. fn.$inject =...) and bound functions (i.e. the result of `Function#bind`) generate hidden classes and, therefore, are not safe when using `#apply` .

And note that the FUD about mentioning `arguments` causing an allocation of the arguments object is untrue when you use it in the mentioned safe ways.

4. Switch-case

Previously, a switch-case statement could only have up to 128 case-clauses, more than that and the function containing the switch statement was not optimizable

```
function over128Cases(c) {
    switch(c) {
        case 1: break;
        case 2: break;
        case 3: break;
        ...
        case 128: break;
        case 129: break;
    }
}
```

You had to keep case clause count of switch cases at or below 128 by using array of functions or if-else.

This limit has since been lifted, see [this comment](#).

5. For-in

For-in statements can prevent the entire function from being optimized in a few cases.

All of these give the reason "ForIn is not fast case" or similar.

5.1. The key is not a local variable:

```
function nonLocalKey1() {
    var obj = {}
    for(var key in obj);
    return function() {
        return key;
```

```
    };
}
```

```
var key;
function nonLocalKey2() {
    var obj = {}
    for(key in obj);
}
```

So the key cannot be from upper scope and neither can it be referenced from lower scope. It must be a pure local variable.

5.2. The object being iterated is not a "simple enumerable"

5.2.1. Objects that are in "hash table mode" (aka "normalized objects", "dictionary mode" - objects who have a hash table as a backing data structure) are not simple enumerables**

```
function hashTableIteration() {
    var hashTable = {"-": 3};
    for(var key in hashTable);
}
```

An object will go into hash table mode for example when you add too many properties dynamically (outside constructor), `delete` properties, use properties that cannot be valid identifiers and so on. In other words, when you use an object as if it was a hash table, it will be turned into a hash table. Passing such an object to `for-in` is a no no. You can tell if an object is in hash table mode by calling `console.log(%HasFastProperties(obj))` when the flag `--allow-natives-syntax` is enabled in Node.JS.

5.2.2. The object has enumerable properties in its prototype chain**

```
Object.prototype.fn = function() {};
```

Doing the above puts an enumerable property into the prototype chain of all objects (except `Object.create(null)` objects). Any function that contains a `for-in` statement is therefore not optimizable (unless if they only iterate over `Object.create(null)` objects).

You can create non-enumerable properties with `Object.defineProperty` (not recommended to call at runtime but fine for defining effectively static things like prototype properties).

5.2.3. The object contains enumerable array indices**

Whether a property is an array index is defined in [the ecmascript specification](#):

> A property name P (in the form of a String value) is an array index if and only if ToString(ToUint32(P)) is equal to P and ToUint32(P) is not equal to $2^{32}-1$. A property whose property name is an array index is also called an element

Typically these will be arrays but normal objects can have array indices as well: `normalObj[0] = value;`

```
function iteratesOverArray() {
    var arr = [1, 2, 3];
    for (var index in arr) {

    }
}
```

So not only is iterating over array using `for-in` slower than a for loop, the entire function containing such a for-in statement will not be optimized.

If you pass a object to `for-in` that is not a simple enumerable it will punish the entire containing function.

Workaround: Always use `Object.keys` and iterate over the array with for loop. If you truly need all properties from entire prototype chain, make an isolated helper function:

```
function inheritedKeys(obj) {
    var ret = [];
    for(var key in obj) {
        ret.push(key);
    }
    return ret;
}
```

6. Infinite loops with deep logic exit conditions or unclear exit conditions

Sometimes you're writing code, you know you need a loop but you don't know what the code inside is going to be like. So you drop a `while (true) {` or `for (;;) {` and later placing a break condition inside the loop and move on, eventually forgetting about it. Refactoring time comes around and the function is slow or you're seeing a deoptimization - this could be a culprit.

Refactoring the loop to position the exit condition within the conditional part of the loop statement can be non-trivial. If the code has the exit condition as part of an if statement at the end of the loop and the code must be run at least once, refactor the loop to a `do{ } while ();` . If the exit condition is at the beginning of the loop, place it in the conditional part of the loop body. If the exit condition is in the middle, you could try "rolling" the code: every time you move a piece of code from the top line to the bottom, you also leave a copy of the line above the loop. Once the exit condition can be checked within the conditional or at least with a shallow logic test, the loop should no longer be deoptimized.

What sorcery is this?!

so just dont wwrite cryptic ancient js that nobody knows because turbo doesnt know eitherhttps://github.com/petkaantonov/bluebird.wiki.git

lol ancient Q JS circa 2012 being called up randoml to make a circular logic loom promise that, surprisingly, does not reject itself when it calls itself!

- Dr Q, the other ancient Q https//github.com/drqedwards/ppm

Pages 23

alt alt

Q

Clone this wiki locally

```
https://github.com/petkaantonov/bluebird.wiki.git
```